Teste de Integração

Herysson R. Figueiredo | Sistemas de Informação

Objetivos da aula

- Compreender por que unidades testadas isoladamente ainda podem falhar quando combinadas.
- Identificar os principais defeitos associados a interfaces, dependências e troca de dados.
- Distinguir integração big-bang de integração incremental.
- Comparar as estratégias incrementais top-down e bottom-up e o papel de stubs e drivers.
- Relacionar teste de integração a regressão, smoke testing e integração de APIs Web.

Fonte: adaptado de MARTINS (2016, Seção 3.3) e PRESSMAN; MAXIM (2021, cap. 19).

Onde estamos?



Agora o foco deixa de ser a unidade isolada e passa a ser a colaboração entre unidades.

Síntese dos níveis de teste com base em PRESSMAN; MAXIM (2021) e MARTINS (2016).

Mapa da aula

01 Fundamentos

Por que integrar e o que observar

02 Estratégias

Big-bang × incremental

03 Top-down

Stubs, largura e profundidade

04 Bottom-up

Clusters e drivers

05 Regressão

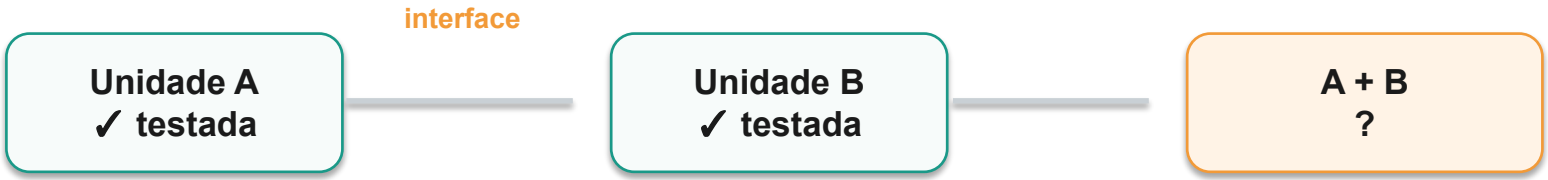
Impactos de novas integrações

06 Smoke + Web

Build estável, API e persistência

Organização didática elaborada a partir de Pressman & Maxim (2021) e Martins (2016).

Da unidade para a integração



Duas unidades corretas isoladamente não garantem uma integração correta.

Fonte: MARTINS (2016, p. 71-75); PRESSMAN; MAXIM (2021, cap. 19).

O problema central

A funciona sozinha

Entradas válidas
Saídas corretas
Testes unitários passam

B funciona sozinha

Contrato próprio
Estado interno correto
Testes unitários passam

A + B falham

Dados incompatíveis
Ordem incorreta
Exceção não tratada

O defeito pode estar na interação — não necessariamente em A ou B isoladamente.

Fonte: MARTINS (2016, Seção 3.3).

O que é teste de integração?

FOCO

Interfaces e comunicação entre unidades, componentes ou subsistemas.

OBJETIVO

Revelar defeitos que surgem quando partes já testadas passam a colaborar.

RESULTADO

Evidências de que contratos, fluxos e dependências funcionam em conjunto.

Integração testa relações entre partes — não apenas as partes em si.

Fonte: MARTINS (2016, p. 71-75); PRESSMAN; MAXIM (2021, cap. 19).

Na integração, o foco muda para a interface



Elaboração didática com base em MARTINS (2016) e PRESSMAN; MAXIM (2021).

O que pode dar errado na integração?

Contrato

Método recebe/retorna algo diferente do esperado.

Dados

Conversão, serialização ou mapeamento incorreto.

Controle

Chamada ocorre na ordem errada ou não ocorre.

Estado

Transação, cache ou estado compartilhado inconsistente.

Erro

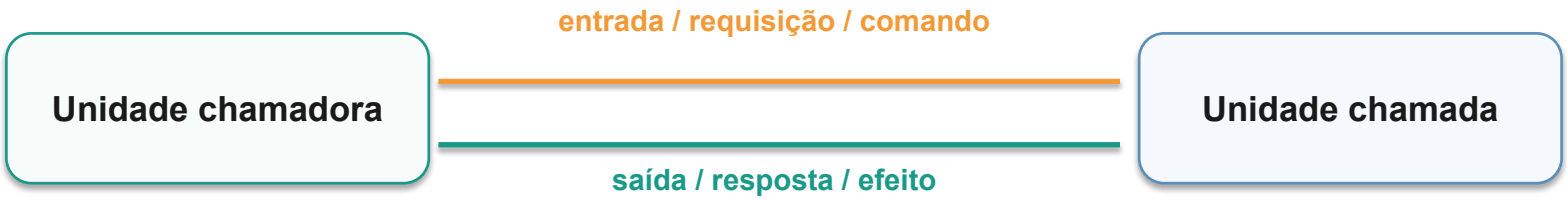
Exceção ou código de erro não é tratado corretamente.

Ambiente

Configuração, credencial ou dependência externa difere.

Exemplos didáticos; conceitos fundamentados em MARTINS (2016, Seção 3.3).

O contrato entre unidades



Integração verifica se as duas partes interpretam o mesmo contrato da mesma maneira.

Elaboração didática com base em princípios de teste de interfaces apresentados por Pressman & Maxim (2021).

Unidade × integração × sistema

Nível	Pergunta principal	Exemplo
Unidade	A classe/método funciona isoladamente?	Service + mocks
Integração	As partes colaboram corretamente?	Service + Repository + DB
Sistema	O produto completo funciona no ambiente?	Aplicação integrada + infraestrutura

A fronteira entre níveis depende do escopo escolhido, mas o objetivo do teste deve ficar explícito.

Fonte: PRESSMAN; MAXIM (2021, sec. 19.1.3) e MARTINS (2016, cap. 3).

O que pode ser integrado?

Métodos / classes
objetos que colaboram

Módulos

unidades da arquitetura

Camadas

Controller ↔ Service ↔ Repository

Banco de dados
ORM ↔ SQL ↔ SGBD

Serviços

API ↔ API / mensageria

“Integração” não significa necessariamente testar o sistema inteiro.

Elaboração didática com base em MARTINS (2016) e PRESSMAN; MAXIM (2021).

Estratégias de integração

BIG-BANG

Várias unidades são combinadas de uma vez e o conjunto é testado posteriormente.

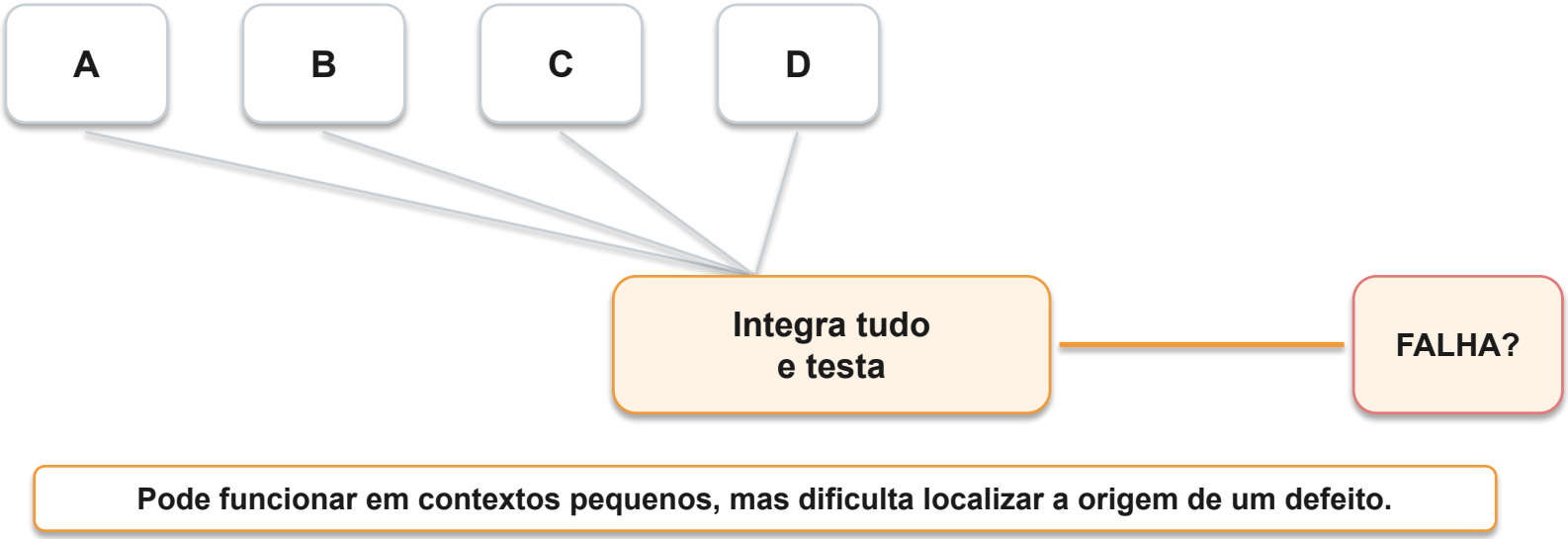
INCREMENTAL

Unidades são incorporadas gradualmente e cada novo incremento é testado.

A diferença central é quando e como as unidades são incorporadas ao conjunto.

Fonte: MARTINS (2016, p. 73-75).

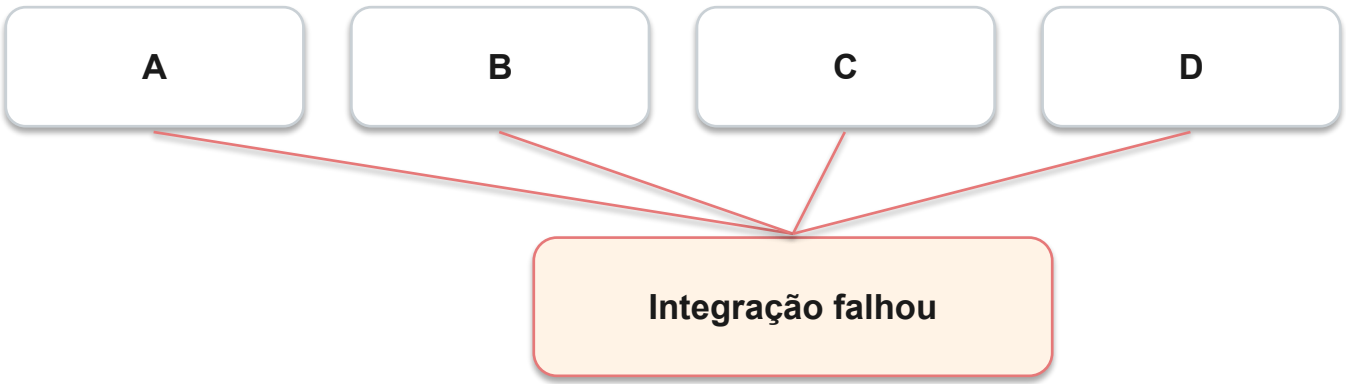
Integração big-bang



Pode funcionar em contextos pequenos, mas dificulta localizar a origem de um defeito.

Fonte: MARTINS (2016, p. 73).

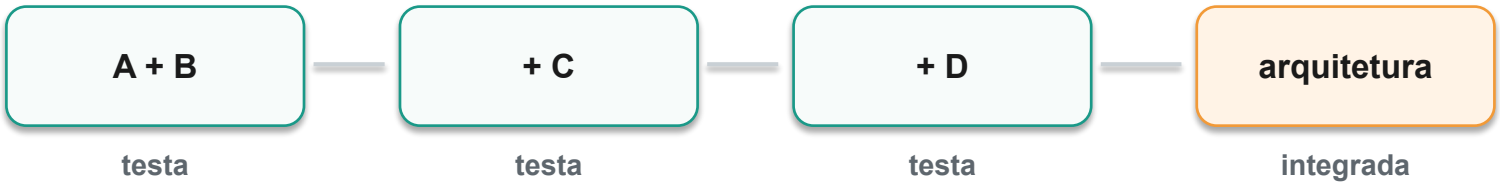
Por que o big-bang dificulta a depuração?



Onde está a causa? Em A, B, C, D ou em alguma das interfaces entre elas?

Fonte: MARTINS (2016, p. 73).

Integração incremental



Se a falha surge após adicionar C, o espaço de investigação fica muito menor.

Fonte: MARTINS (2016, p. 73-75).

Integração top-down (descendente)

Começa no topo

Seleciona-se um módulo principal e integra-se progressivamente as unidades abaixo dele.

Pode avançar

Em largura ou em profundidade, de acordo com a arquitetura e a estratégia escolhida.

Usa stubs

Unidades inferiores ainda indisponíveis podem ser substituídas temporariamente.

A estrutura da arquitetura orienta a ordem de integração.

Fonte: MARTINS (2016, p. 73-74); Pressman & Maxim (2021).

Top-down: exemplo de arquitetura

Comece por U1 e incorpore módulos abaixo.

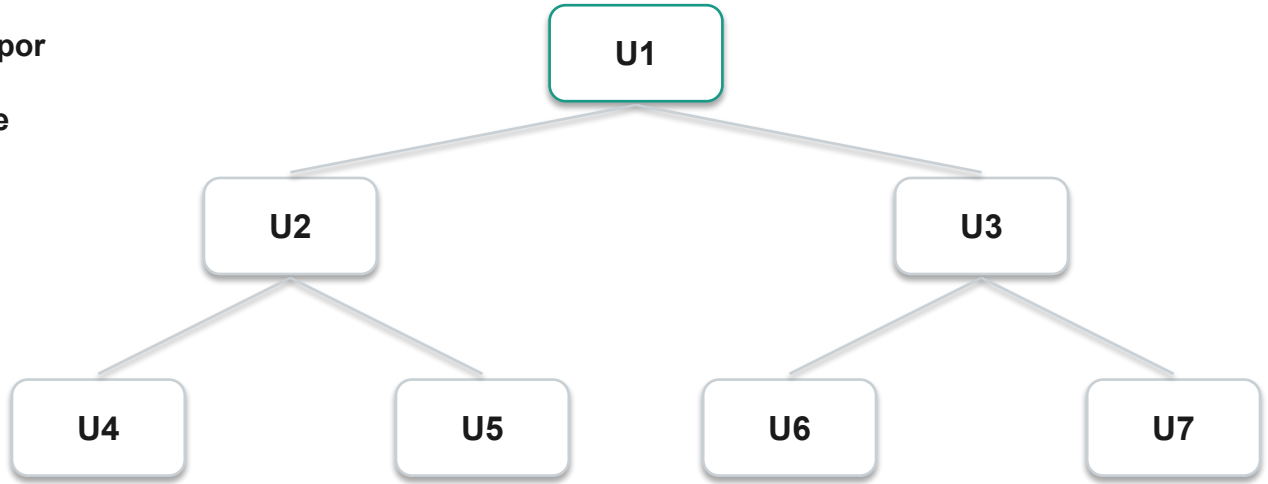
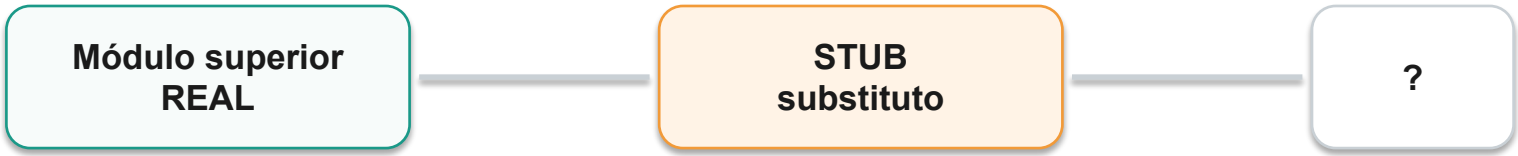


Figura didática baseada na estratégia apresentada em MARTINS (2016, p. 73-74).

Stub: substituto temporário de uma unidade chamada



O stub fornece respostas controladas enquanto o componente inferior real ainda não está disponível ou ainda não foi integrado.

Útil no top-down; porém, o stub não substitui o teste posterior com a unidade real.

Fonte: MARTINS (2016, p. 69-74).

Top-down: largura × profundidade

LARGURA

Integra primeiro os módulos do mesmo nível arquitetural.

Ex.: U1 → U2 e U3 antes de descer.

PROFUNDIDADE

Segue um ramo funcional até níveis inferiores antes de voltar a outro ramo.

Ex.: U1 → U2 → U4.

Não existe uma ordem universal: a arquitetura e o risco orientam a escolha.

Fonte: MARTINS (2016, p. 73-74).

Top-down: exemplo de arquitetura

Comece por U1 e incorpore módulos abaixo.

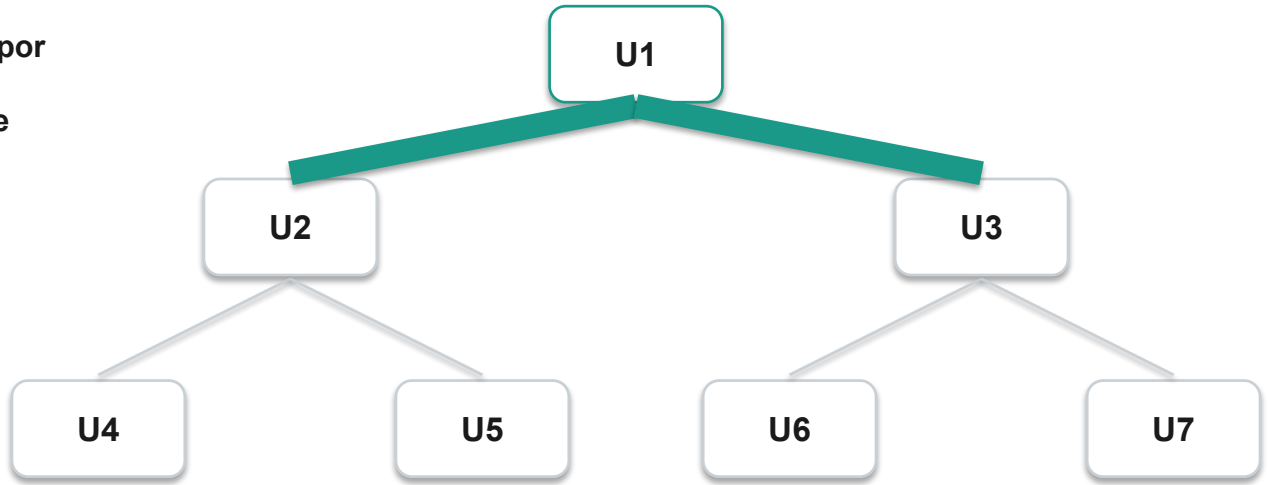


Figura didática baseada na estratégia apresentada em MARTINS (2016, p. 73-74).

Top-down: exemplo de arquitetura

Comece por U1 e incorpore módulos abaixo.

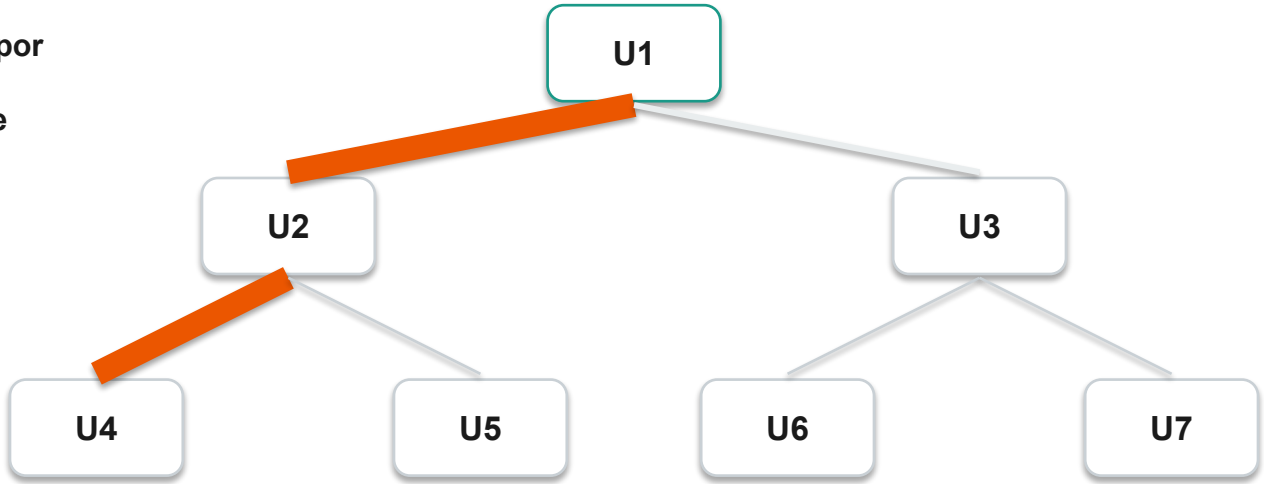


Figura didática baseada na estratégia apresentada em MARTINS (2016, p. 73-74).

Integração bottom-up (ascendente)

Começa embaixo

Unidades inferiores e altamente dependentes são reunidas em clusters.

Testa clusters

Cada cluster é exercitado antes de ser incorporado a módulos superiores.

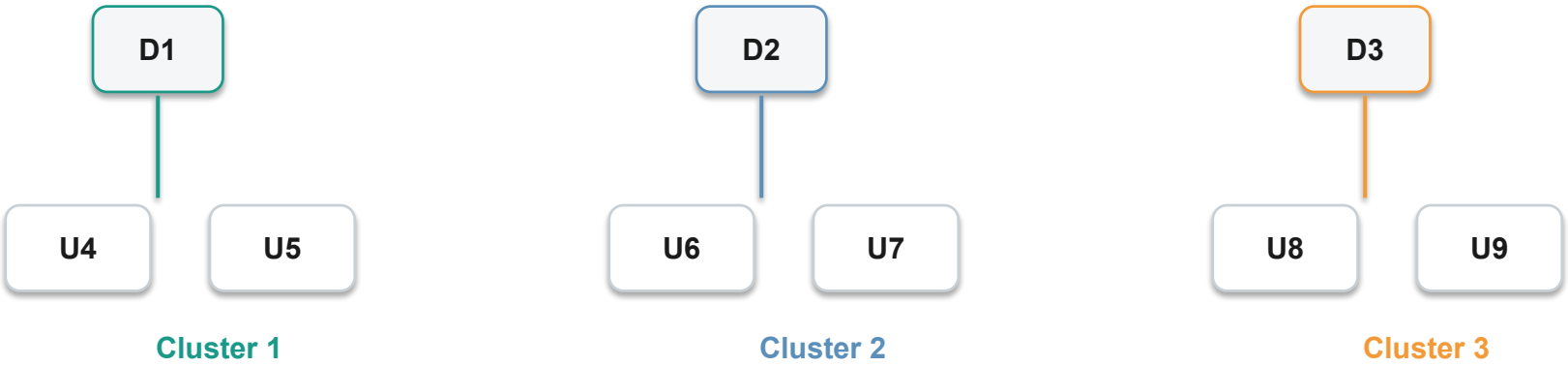
Usa drivers

Drivers simulam módulos superiores para disparar o comportamento do cluster.

À medida que sobe, drivers são removidos e os módulos reais assumem seu lugar.

Fonte: MARTINS (2016, p. 74-75).

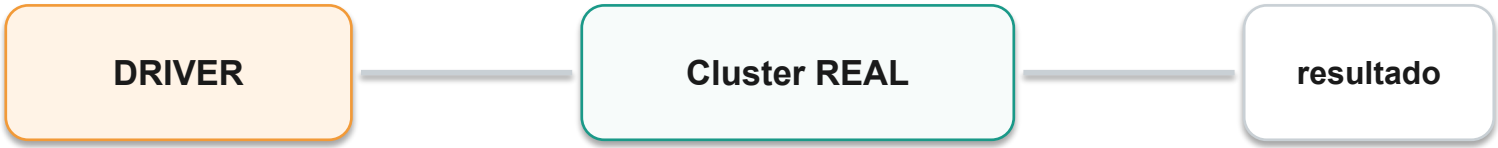
Bottom-up: exemplo com clusters



O driver “chama” o cluster enquanto o módulo superior real ainda não participa do teste.

Figura didática baseada em MARTINS (2016, p. 74-75).

Driver: substituto temporário de uma unidade chamadora

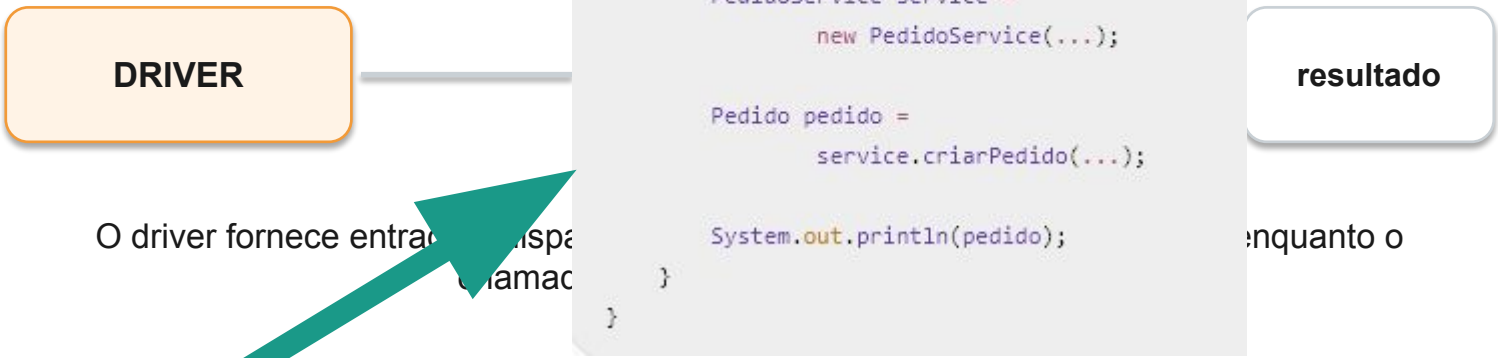


O driver fornece entradas, dispara a execução e observa saídas do cluster enquanto o chamador real ainda não foi integrado.

unidades auxiliares de teste:
Driver: quem chama a unidade testada
Stub: quem é chamado pela unidade testada

Fonte: MARTINS (2016, p. 69-75).

Driver: substituto ter de chamadora



O driver fornece entra ispa enquanto o
amac

unidade auxiliares de teste:
Driver: quem chama a unidade testada
Stub: quem é chamado pela unidade testada

Fonte: MARTINS (2016, p. 69-75).

Driver: substituto temporário de uma unidade chamadora

Queremos testar `PedidoController` + `PedidoService`, mas o serviço externo de pagamento ainda não está disponível.

```
public class PagamentoServiceStub {

    public boolean autorizarPagamento(
            double valor) {

        return true;
    }
}
```

resultado

O driver fornece entrad[illegible]spa[illegible]uster enquanto o
c[illegible]amad[illegible]

unidades auxiliares de teste:
Driver: quem chama a unidade testada
Stub: quem é chamado pela unidade testada

Fonte: MARTINS (2016, p. 69-75).

Top-down × bottom-up

Aspecto	Top-down	Bottom-up
Início	módulos superiores	módulos inferiores
Substitutos mais comuns	stubs	drivers
Organização	árvore: largura/profundidade	clusters dependentes
Vantagem didática	fluxos principais cedo	infraestrutura real cedo

As duas estratégias são incrementais; mudam principalmente a ordem e os elementos temporariamente substituídos.

Fonte: MARTINS (2016, p. 73-75).

Como escolher a estratégia de integração?

Arquitetura

quais módulos dependem de quais?

Risco

quais interfaces são mais críticas?

Disponibilidade

quais componentes já estão prontos?

Diagnóstico

quão fácil é isolar uma falha?

Custo

quanto custa manter stubs/drivers?

A melhor estratégia é a que reduz risco e facilita observar a origem dos defeitos.

Síntese didática com base em PRESSMAN; MAXIM (2021) e MARTINS (2016).

Teste de regressão

Reexecutar testes já realizados para verificar se uma nova integração ou modificação introduziu efeitos indesejados.



Fonte: MARTINS (2016, p. 75-76), com base em Pressman.

Por que a regressão acompanha a integração?

Novos fluxos

Uma integração cria novas rotas de dados e de controle entre unidades.

Efeitos colaterais

Partes antes estáveis podem ser alcançadas de uma forma nova.

Risco acumulado

Cada incremento modifica o sistema integrado e pode reativar defeitos.

Não basta testar apenas o componente recém-adicionado.

Fonte: MARTINS (2016, p. 75-76).

Quais testes entram na regressão?

1. **Representativos** — Casos que exercitam as funcionalidades principais.
2. **Áreas impactadas** — Casos focados em partes potencialmente afetadas pela mudança.
3. **Novo / modificado** — Casos específicos para a unidade recém-integrada ou alterada.

Executar tudo sempre pode ser caro; selecionar bem a regressão exige conhecimento do sistema.

Fonte: MARTINS (2016, p. 75-76), com base em Pressman.

Regressão completa ou seletiva?

COMPLETA

Reexecuta toda a suíte disponível.

Maior abrangência, maior tempo e custo.

SELETIVA

Prioriza casos representativos, partes impactadas e mudanças recentes.

Menor custo, exige boa análise de impacto.

A seleção é uma decisão de risco, custo e tempo — não apenas de conveniência.

Fonte: MARTINS (2016, p. 75-76).

Smoke testing — teste de fumaça

Uma build recém-integrada passa por um conjunto curto de testes críticos para verificar se está estável o suficiente para testes mais profundos.

Objetivo

Encontrar rapidamente erros bloqueadores.

Frequência

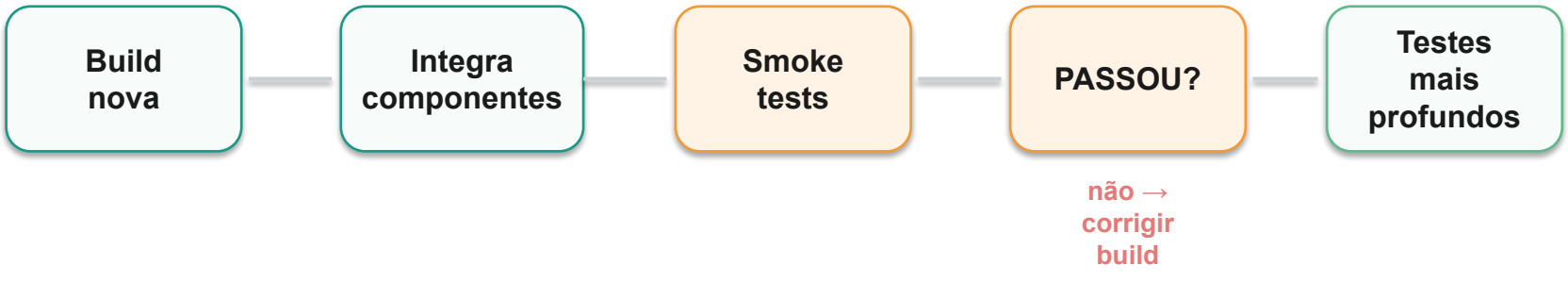
Pode acompanhar builds frequentes / integração contínua.

Escopo

Não é exaustivo: verifica funções essenciais.

Fonte: PRESSMAN; MAXIM (2021, cap. 19).

Fluxo típico de smoke testing



Uma build que falha no smoke test não está pronta para a bateria completa.

Síntese baseada em PRESSMAN; MAXIM (2021, cap. 19).

Smoke test ≠ regressão

SMOKE

Pergunta: “A build está minimamente estável?”

Poucos testes críticos e rápidos.

REGRESSÃO

Pergunta: “A mudança quebrou algo que já funcionava?”

Reexecução planejada de testes anteriores.

As duas estratégias podem coexistir na mesma pipeline de testes.

Elaboração didática com base em PRESSMAN; MAXIM (2021) e MARTINS (2016).

Integração em aplicações modernas



Cada fronteira cria contratos e riscos de integração diferentes.

Elaboração didática com base em Pressman & Maxim (2021) e Martins (2016).

Integração em uma API REST



O teste pode revelar falhas em JSON ↔ objeto, validação, regras, persistência e resposta HTTP — tudo no mesmo fluxo.

Integração com banco de dados

Mapeamento

Entidade ↔ tabela
Tipos e colunas
Chaves e relacionamentos

Persistência

INSERT / UPDATE / DELETE
Consultas e filtros
Dados realmente gravados

Transação

Commit / rollback
Consistência
Efeitos parciais

D B

Mock de Repository não verifica se JPA, SQL e o SGBD realmente trabalham juntos.

Um **mock de Repository** é uma versão simulada de um `Repository` usada durante o teste para evitar acesso real ao banco de dados.

Síntese didática; MARTINS (2016) destaca a importância de verificar dados armazenados e trocados entre componentes.

Integração com serviços externos



Teste com substituto

Bom para cenários controlados e falhas simuladas.

Teste real

Necessário para validar contratos e interoperabilidade reais.

Cuidado

Latência, indisponibilidade e dados externos reduzem repetibilidade.

Elaboração didática com base nos princípios de teste de interfaces.

Dados e ambiente de teste

DADOS

Conhecidos
Repetíveis
Representativos
Sem depender de produção

AMBIENTE

Configuração controlada
Banco de teste
Credenciais próprias
Versões conhecidas

LIMPEZA

Estado inicial conhecido
Remover dados criados
Evitar dependência entre testes

Um teste de integração útil precisa ser reproduzível.

Fonte: princípios de estratégia de teste apresentados em Pressman & Maxim (2021) e Martins (2016).

Independência entre testes de integração

Um teste não deveria passar ou falhar apenas porque outro teste foi executado antes.

1. **Preparar**
   criar estado necessário
2. **Executar**
   realizar a interação
3. **Verificar**
   observar resposta + efeitos
4. **Limpar**
   restaurar estado

Repetibilidade é parte da confiabilidade da suíte de testes.

Elaboração didática.

Falhas comuns de integração em APIs

HTTP

status incorreto / método errado

JSON

campo, tipo ou nome incompatível

Validação

regra não aplicada entre camadas

Persistência

dado não salvo ou salvo parcialmente

Erro

exceção interna vira resposta inadequada

Estado

ordem dos requests altera o resultado

Estudo de caso da prática: Menu-Back-End

STACK

Java 17+
Spring Boot
Spring Data JPA
MySQL
Maven

DOMÍNIO

API REST para cadastro e gerenciamento de produtos de uma hamburgueria.

OPERAÇÕES

GET
POST
PUT
DELETE
/produtos

Na aula prática, o Postman será usado para executar e automatizar os casos sobre a API real.

Fonte: repositório github.com/Herysson/Menu-Back-End, acesso em 09/11/2026.

As técnicas anteriores continuam valendo

Caixa-preta

Equivalência
Valor limite
Tabela de decisão
Estados / casos de uso

Caixa-branca

Nós / ramos
Condições
Fluxo de dados
Caminhos

Integração

Interfaces
Contratos
Persistência
Fluxos entre partes

Ferramenta

Postman
Collection
Scripts

A ferramenta executa os casos; quem escolhe bons casos são as técnicas de teste.

Síntese das aulas anteriores aplicada ao teste de integração.

Ponte para a aula prática com Postman

1 Subir API

2 Criar Collection

3 CRUD completo

4 Assertions

5 Técnicas de teste

6 Regressão / smoke

Objetivo da prática: transformar técnicas de projeto de teste em uma suíte executável sobre uma API integrada.

A prática será disponibilizada em material separado (.md).

Atividade de discussão

Para cada situação, classifique o teste como unitário, integração ou sistema — e justifique.

- **A** Service com Repository mockado.
- **B** Repository real gravando no banco de teste.
- **C** Controller + Service + Repository + banco.
- **D** Frontend + API + banco + serviço externo em ambiente completo.

Elaboração didática.

Resumo

- Teste de integração procura defeitos nas interfaces e interações entre unidades.
- A abordagem incremental facilita localizar defeitos em comparação ao big-bang.
- Top-down integra a partir do topo e costuma usar stubs; bottom-up integra clusters inferiores e usa drivers.
- Regressão verifica se novas integrações afetaram comportamentos que já funcionavam.
- Smoke testing verifica rapidamente se uma nova build está estável o suficiente para testes mais profundos.
- Em APIs, integrar significa exercitar contratos, camadas, persistência e respostas reais.

Síntese baseada nas referências utilizadas na aula.

Próximo passo: prática com Menu-Back-End + Postman



Teoria nos slides; laboratório executável e passo a passo no .md.

Prática planejada como tutorial separado em formato Markdown.

Referências

Referências consultadas para a elaboração desta aula.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

MARTINS, Marcos Danilo Chiodi. Testes de software. 1. ed. Rio de Janeiro: SESES, 2016.

HERYSSON FIGUEIREDO. Menu-Back-End. GitHub. Repositório utilizado como estudo de caso da prática: github.com/Herysson/Menu-Back-End.